

Índice de contenido

1. Arquitectura Base y el Bucle Principal.....	2
2. Ventanas Secundarias y Herencia (VentanaBase).....	2
3. Widgets Especializados y Formularios.....	3
4. El Sistema de Ordenación y Eventos.....	3
5. Programación Orientada a Objetos (POO) en Interfaces.....	4
6. El Sistema de Geometría: Grid vs. Pack.....	4
7. Herencia: El "Plano Maestro" (VentanaBase).....	5
8. Visualización Avanzada con Treeview.....	5
9. El Orquestador: main.py.....	6
10. La Plantilla Maestra: VentanaBase.py.....	6
11. Implementación Específica: VentanaComprados.py.....	7

Aquí veremos con ejemplos de código reales, como se realiza un programa con Tkinter, enseñando las partes más importantes y explicando su funcionalidad y del ¿Por qué? Se hizo así.

Ahora pondremos en práctica lo aprendido tras consultar el manual de Tkinter y podremos resolver posibles preguntas referentes a un uso real y eficiente.

1. Arquitectura Base y el Bucle Principal

Toda aplicación de Tkinter requiere una raíz. En el código de `main.py`, esto se gestiona mediante la clase `AppPrincipal`. A diferencia de los scripts lineales, el uso de clases permite mantener un estado organizado del programa.

El Punto de Entrada

Python

```
if __name__ == "__main__":  
    root = tk.Tk() # Creación de la ventana raíz  
    app = AppPrincipal(root)  
    root.mainloop() # El bucle que mantiene la app "escuchando"
```

Utilidad: `mainloop()` es el motor que permite que los botones reaccionen y las ventanas no se cierren instantáneamente al ejecutarse.

2. Ventanas Secundarias y Herencia (VentanaBase)

Para evitar repetir código (como la configuración de tablas o el tamaño de ventana), el programa utiliza la **herencia**. `VentanaBase` define el comportamiento común de todas las subventanas (Libros, Deseados, Prestados).

El uso de `Toplevel` y `Treeview`

- **Toplevel:** Se usa para crear ventanas hijas independientes de la principal.
- **Treeview:** Es el widget avanzado de `ttk` para mostrar datos en formato de tabla.

Python

```
self.window = tk.Toplevel(master)  
# Crea una nueva ventana sobre la raíz  
  
self.tree = ttk.Treeview(self.window, columns=self.columns,  
show="headings")
```

3. Widgets Especializados y Formularios

En los archivos como `VentanaComprados.py`, se utilizan widgets más allá del simple `Entry` para mejorar la experiencia del usuario:

- **Spinbox:** Ideal para valores numéricos (como el número de páginas).
- **Checkbutton:** Para valores binarios (¿Leído? Sí/No).
- **Messagebox:** Para validaciones y confirmaciones antes de guardar o borrar.

Ejemplo de Lógica de Formulario:

Python

```
def abrir_formulario(self, datos=None):
    f = tk.Toplevel(self.window)
    f.grab_set()
    # Bloquea la interacción con la ventana de atrás hasta cerrar esta

    tk.Label(f, text="Páginas").pack()
    e_p = tk.Spinbox(f, from_=0, to=9999)
    # Widget especializado
    e_p.pack()
```

4. El Sistema de Ordenación y Eventos

El programa implementa funciones avanzadas que hacen que la interfaz se sienta "viva":

1. **Ordenamiento Dinámico:** Al hacer clic en la cabecera de una columna en el `Treeview`, los datos se reordenan mediante la función `ordenar_columna` en `VentanaBase.py`.
2. **Event Binding:** Se usa `.bind("<Double-1>", ...)` para que, al hacer doble clic en una fila, se abra automáticamente el formulario de edición.
3. **Gestión de Instancias:** En `main.py`, el método `controlar_ventana` asegura que no se abran múltiples copias de la misma ventana:

Python

```
if ventana is None or not ventana.window.winfo_exists():
    self.ventanas_abiertas[clave] = clase_ventana(self.root, self.db)
else:
    self.ventanas_abiertas[clave].window.lift()

# La trae al frente si ya existe
```

5. Programación Orientada a Objetos (POO) en Interfaces

El uso de clases no es opcional en aplicaciones profesionales. En `main.py`, la clase `AppPrincipal` actúa como el **Controlador Maestro**.

- **¿Por qué usar clases?:** Permiten encapsular variables (como `self.ventanas_abiertas`) para que persistan mientras la aplicación esté en ejecución.
- **Gestión de Ventanas Únicas:** Para evitar que el usuario abra diez veces la misma ventana, se utiliza un diccionario que rastrea si la ventana existe.

Python

```
# Si la ventana no existe o fue cerrada, se crea una nueva instancia
if ventana is None or not ventana.window.winfo_exists():
    self.ventanas_abiertas[clave] = clase_ventana(self.root, self.db)
```

- **winfo_exists():** Este método de Tkinter verifica si el widget aún está en memoria.
- **lift() y focus_force():** Si la ventana ya existe, estas funciones la traen al frente de la pantalla para llamar la atención del usuario.

6. El Sistema de Geometría: Grid vs. Pack

El manual base indica que `grid` es superior para formularios. LibroPy combina ambos para maximizar la eficiencia.

- **Grid (La Rejilla):** Se utiliza en los formularios de edición (`VentanaComprados.py`, `VentanaDeseados.py`). Funciona como una tabla de Excel donde indicamos la fila (`row`) y columna (`column`).
- **Sticky:** Define la alineación. `sticky="e"` (East) alinea el texto a la derecha, mientras que `sticky="nsew"` hace que un botón se estire para ocupar todo el espacio disponible.
- **Pack (El Empaquetador):** Se usa para componentes que deben apilarse de arriba hacia abajo, como el `Treewiew` y los marcos de botones (`btn_frame`).

7. Herencia: El "Plano Maestro" (VentanaBase)

La herencia permite que `VentanaComprados`, `VentanaDeseados` y `VentanaPrestados` compartan una misma estructura sin repetir código.

El Método `__init__` y `super()`

Cuando una ventana hija se crea, llama a `super().__init__()`. Esto ejecuta la configuración de `VentanaBase.py`, la cual:

1. Crea un nuevo nivel de ventana (`TopLevel`).
2. Define el tamaño estándar (950x550).
3. Configura el sistema de ordenación de columnas.

8. Visualización Avanzada con Treeview

El `Treeview` es el widget más complejo de este programa. No solo muestra texto, sino que gestiona interacciones de usuario.

- **Ordenación Dinámica:** En `VentanaBase.py`, la función `ordenar_columna` convierte los datos de la tabla en una lista de Python, los ordena (manejando números y monedas con `safe_float`) y vuelve a posicionarlos en la interfaz.
- **Binding de Eventos:**
 - `<Double-1>`: Detecta un doble clic para editar.
 - `<Button-1>`: Se usa para detectar clics en áreas vacías y quitar la selección actual, mejorando la usabilidad.

9. El Orquestador: `main.py`

Este es el punto de entrada. Su lógica principal es el **Control de Instancias**.

Evitar Ventanas Duplicadas

Python

```
def controlar_ventana(self, clave, clase_ventana):
    ventana = self.ventanas_abiertas[clave]

    # Comprobación de existencia real en memoria
    if ventana is None or not ventana.window.wininfo_exists():
        self.ventanas_abiertas[clave] = clase_ventana(self.root, self.db)
    else:
        # Si ya existe, se "despierta" y se trae al frente
        self.ventanas_abiertas[clave].window.lift()
        self.ventanas_abiertas[clave].window.focus_force()
```

- **Lógica:** `wininfo_exists()` es crucial. Sin esto, si un usuario cierra una ventana y pulsa el botón de nuevo, el programa intentaría acceder a un objeto destruido y daría error. Aquí, si detecta que no existe, crea una nueva instancia de la clase correspondiente.

10. La Plantilla Maestra: `VentanaBase.py`

Aquí aplicamos la **Herencia**. Todas las ventanas de la app son similares (tienen una tabla y botones), por lo que definimos esa lógica una sola vez.

Configuración Dinámica de Tablas

Python

```
def configurar_tabla(self):
    # Se crea el widget Treeview
    self.tree = ttk.Treeview(self.window, columns=self.columnas,
                             show="headings")
    for col in self.columnas:
        # Se asigna una función de ordenado a cada cabecera
        self.tree.heading(col, text=col, command=lambda c=col:
            self.ordenar_columna(c))
    self.tree.column(col, width=100, anchor="center")
    self.tree.pack(fill="both", expand=True, padx=15, pady=5)
```

- **Lógica:** Usar un bucle `for` para las columnas permite que esta misma clase sirva para una tabla de 3 columnas o una de 10 sin cambiar el código.
- **Lambda:** El uso de `lambda c=col` es un truco avanzado de Python para "congelar" el valor de la columna actual y pasárselo a la función de ordenado cuando el usuario haga clic.

11. Implementación Específica: `VentanaComprados.py`

Este archivo hereda de `VentanaBase` y añade la lógica de negocio particular para los libros comprados.

La Dualidad del Formulario (Añadir/Editar)

Python

```
def abrir_formulario(self, datos=None):
    f = tk.Toplevel(self.window)
    f.grab_set() # Convierte la ventana en MODAL (bloquea la de atrás)

    # ... creación de campos (Entry, Spinbox) ...

    if datos:
        # Lógica de EDICIÓN: Rellenar campos con lo que ya existe
        e_t.insert(0, datos[1])
        v_l.set(1 if datos[5] == "Sí" else 0)
```

- **Lógica:** El parámetro `datos=None` permite que la misma función sirva para dos cosas. Si pasas datos, los campos se pre-rellenan; si no, aparecen vacíos para un nuevo registro.
- **`grab_set()`:** Es una técnica de UX (Experiencia de Usuario) fundamental para evitar que el usuario modifique la tabla principal mientras tiene un formulario de edición abierto, lo cual causaría conflictos de datos.

El Evento de Selección Inteligente

Python

```
def validar_y_editar(self, event):
    item_id = self.tree.identify_row(event.y)
# Detecta dónde cayó el clic
    if item_id:
        self.tree.selection_set(item_id)
        # Selecciona la fila
        valores = self.tree.item(item_id)['values']
        # Extrae la información
        self.abrir_formulario(valores)

    # Abre el editor con esos datos
```

- **Lógica:** `identify_row(event.y)` es más preciso que simplemente pedir la selección actual, ya que asegura que el usuario realmente hizo doble clic sobre una fila con datos y no en el espacio en blanco de la tabla.